

Lenguajes de Programación

Práctica 2: EAB

1 Introducción

Considera la siguiente extensión al lenguaje de expresiones aritméticas visto en clase:

```
type ID = String

data EAB = Num Int | Var ID | Bool Bool
         | Suma EAB EAB | Prod EAB EAB
         | Suc EAB | Pred EAB
         | Not EAB
         | If EAB EAB EAB
         | IsZero EAB
         | Lt EAB EAB | Gt EAB EAB | Eq EAB EAB
         | Let ID EAB EAB
         deriving (Eq)

type Env = [(ID, EAB)]
```

La practica consiste en definir las siguientes funciones:

1. Instancia la clase `Show` para el tipo de dato `EAB`. (1 pto)

```
{- Ejemplo -}
> show (Let x (Bool True) (If (Var x) (Suma (Num 5) (Num 3)) (Prod (Suc (Num 2))
    (Pred (Num 1)))))
> (Let x = true in if x then (5+3) else (suc(2)*pred(1)))

> show (Not (And (Gt (Num 10) (Num 25)) (Eq (Suma (Num 10) (Num 5)) (Num 11))))
> not ((10>25)&&((10+5)==11))
```

2. Define la función `evalEnv env` e que devuelve el valor resultante de evaluar la expresión `EAB` dada con el ambiente `env`. (2 pto)

```
eval :: Env -> EAB -> Either Int Bool

{- Ejemplo -}
> evalEnv [("x", Num 2)] (Sum (Num 5) (Prod (Num 10) (Var "x")))
> Left 25

> evalEnv [("x", Num 2)] (Let "a" (Num 10) (Let "b" (Num 20) (Sum (Var "a") (Sum
    (Var "b") (Var "x")))))
> Left 32
```

3. Define la función `sust x e1 e2` que sustituye las apariciones libres de la variable `x` por `e1` en la expresión `e2`. No deben implementar α -equivalencia, en su lugar deben mostrar un error informativo ("Error: Captura de Variable libre") (1 pto)

```
sust :: ID -> EAB -> EAB -> EAB

{- Ejemplo -}
> sust "x" (Num 10) (Sum (Var "x") (Num 5))
> (10+5)

> sust "x" (Num 5) (Let "x" (Num 2) (Sum (Var "x") (Var "y")))
> (Let x=2 in ("x"+"y"))
```

2 Semántica Dinámica

1. Utilizando la función `sust` define la función `evalStep e` que cumple la relación `evalStep e = e' sys e  $\rightarrow$  e'`. (1 pto)

```
evalStep :: EAB -> EAB

{- Ejemplo -}
> evalStep (Sum (Sum (Num 10) (Num 10)) (Prod (Num 10) (Num 15)))
> Sum (Num 20) (Prod (Num 10) (Num 15))

> evalStep (Let x (Gt (Num 5) (Num 6)) (If x (Num 6) (Num 0)))
> Let x (Bool False) (If x (Num 6) (Num 0))
```

2. Utilizando la función `evalStep e`, define la función `evalDin e` que cumple la relación `evalDin e = e' sys e  $\rightarrow^*$  e'` con `e'` un estado bloqueado. (1 pto)

```
evalDin :: EAB -> EAB

{- Ejemplo -}
> evalDin (Prod (Sum (Num 10) (Num 5)) (Num 5))
> Num 75

> evalDin (Sum (And (Bool True) (Bool False)) (Num 5))
> Sum (Bool False) (Num 5)
```

3. Define la función `isValid` que indica si `evals e = e' sys e  $\rightarrow^*$  e'` con `e'` un valor. (1 pto)

```
isValid :: EAB -> Bool

{- Ejemplo -}
> isValid (Lt (Sum (Num 5) (Num 5)) (Sum (Num 7) (Prod (Num 7) (Num 8))))
> True

> isValid (Eq (Sum (Num 5) (Num 5)) (Gt (Num 7) (Num 8)))
> False
```

3 Semántica Estática

Considera la siguiente definición de tipos y contexto para el tipo de dato EAB

```
type Ctx = [(ID, Type)]  
data Type = Nat | Boolean
```

1. Define la función `typeEAB ctx e` que recibe un contexto Γ , una expresión *EAB* *e* y cumple la relación `typeEAB ctx e = t` syss $\Gamma \vdash e : T$. En caso contrario, reportar el error de tipado. (2 pto)

```
typeEAB :: Ctx -> EAB -> Type  
  
{- Ejemplo -}  
> typeEAB [("y",Nat)] (Let x (Num 5) (Gt (Sum (Var "y") (Num 5)) (Var "x")))  
> Boolean  
  
> typeEAB [] (Prod (Num 5) (Not (Num 5)))  
> Expected Boolean: (Num 5)
```

2. Define la función `evalEst e` que cumple la relación `evalEst e = e'` syss $e \rightarrow^* e'$ con el tipado de *e* correcto. En caso contrario, reportar el error de tipado. (1 pto)

```
evalEst :: EAB -> Either Int Bool  
  
{- Ejemplo -}  
> evalEst (Eq (Sum (Num 5) (Num 9)) (Num 13))  
> Right False  
  
> evalEst (Sum (Not (Bool True)) (Num 10))  
> Expected Number: (Not (Bool True))
```